

JAVASCRIPT

1. What is JavaScript?

JavaScript is a high-level, interpreted programming language used to create dynamic and interactive web applications.

It runs in browsers and also on servers using Node.js.

It is mainly used for frontend and backend development.

2. Why is JavaScript single-threaded?

JavaScript is single-threaded because it executes one task at a time using a single call stack.

It avoids complexity by maintaining a single execution flow.

For handling multiple tasks, it uses the event loop and asynchronous APIs.

3. What are the data types in JavaScript?

JavaScript has two categories of data types: primitive and non-primitive.

Primitive types include String, Number, Boolean, Null, Undefined, BigInt, and Symbol.

Non-primitive types include Objects, Arrays, and Functions.

4. Difference between primitive and non-primitive data types?

Primitive data types store single values and are immutable.

Non-primitive data types store collections of values and are mutable.

Examples: Primitive - String, Number; Non-primitive - Object, Array.

5. What is typeof operator?

typeof is a JavaScript operator used to check the data type of a value.

It returns the data type as a string.

Example: `typeof 10` returns "number".

6. Difference between null and undefined?

undefined means a variable is declared but no value is assigned.

null represents an intentionally empty value assigned by the developer.

`typeof undefined` is "undefined" and `typeof null` is "object".

7. Difference between let, var, and const?

var is function-scoped and can be redeclared and updated.

let is block-scoped and can be updated but not redeclared in the same scope.
const is block-scoped and cannot be reassigned after declaration.

8. What is hoisting?

Hoisting is JavaScript's behavior of moving declarations to the top of their scope.
Variables and functions are processed before code execution.
Function declarations can be called before they are defined.

9. What is Temporal Dead Zone (TDZ)?

TDZ is the period when let and const variables exist but cannot be accessed.
It starts from the beginning of the block until the variable declaration line.
Accessing them during TDZ gives a ReferenceError.

10. What is scope in JavaScript?

Scope defines the accessibility of variables in a program.
It decides where a variable can be used and accessed.
JavaScript has global, function, and block scope.

11. Difference between global, function, and block scope?

Global scope variables can be accessed throughout the program.
Function scope variables are accessible only inside a function.
Block scope variables are accessible only inside {} using let and const.

12. What is type coercion?

Type coercion is the automatic or manual conversion of one data type into another.
JavaScript converts values when performing operations between different types.
Example: "5" + 2 returns "52".

13. Difference between == and ===?

== compares only values and allows type conversion.
=== compares both value and data type without conversion.
=== is preferred because it gives more predictable results.

14. What are truthy and falsy values?

Truthy values are values that are treated as true in boolean conditions.
Falsy values are treated as false.
Falsy values include false, 0, "", null, undefined, and NaN.

15. What is NaN?

NaN stands for "Not a Number".

It represents an invalid or unsuccessful mathematical operation.
Example: `Number("abc")` returns NaN.

JavaScript Functions (3-Line Interview Answers)

1. What is a function?

A function is a reusable block of code that performs a specific task.
It executes only when it is called (invoked).
Functions help avoid code repetition and improve readability.

2. Function declaration vs Function expression?

A function declaration is defined using the `function` keyword and is hoisted.
A function expression stores a function inside a variable and is not fully hoisted.
Function expressions can be anonymous or named.

```
// Function Declaration  
function greet() { }
```

```
// Function Expression  
const greet = function() { };
```

3. What are arrow functions?

Arrow functions are a shorter way to write functions, introduced in ES6.
They use the `=>` syntax instead of the `function` keyword.
They do not have their own `this` value.

```
const add = (a, b) => a + b;
```

4. Difference between normal and arrow functions?

Normal functions have their own `this` and can be used as constructors.

Arrow functions inherit `this` from the surrounding scope.

Arrow functions are shorter and cannot be used with `new`.

5. What is a callback function?

A callback function is a function passed as an argument to another function.

It is executed after a specific task is completed.

Callbacks are commonly used in asynchronous programming.

```
function greet(name, callback){  
    callback(name);  
}
```

6. What are first-class functions?

In JavaScript, functions are first-class citizens.

They can be assigned to variables, passed as arguments, and returned from other functions.

This makes JavaScript flexible and powerful.

7. What are higher-order functions?

A higher-order function is a function that takes another function as an argument or returns a function.

Examples include `map()`, `filter()`, and `reduce()`.

They help write cleaner and reusable code.

8. What is a pure function?

A pure function always returns the same output for the same input.

It does not modify external variables or produce side effects.

Pure functions are predictable and easy to test.

9. What is an IIFE?

IIFE stands for Immediately Invoked Function Expression.

It runs immediately after it is created.

It helps avoid polluting the global scope.

```
(function(){  
    console.log("Hello");  
})();
```

10. What are rest parameters?

Rest parameters allow a function to accept multiple arguments as an array.

They are represented using three dots (...).

They must be the last parameter in the function.

```
function sum(...nums){  
    return nums;  
}
```

11. What is the spread operator?

The spread operator (...) expands elements of an array or object.

It is used for copying, merging, or passing multiple values.

It makes code shorter and cleaner.

```
const arr = [1,2,3];  
const copy = [...arr];
```

12. What is recursion?

Recursion is a technique where a function calls itself.

It continues until a base condition is met.

It is commonly used for problems like factorial, Fibonacci, and tree traversal.

```
function fact(n){  
    if(n === 1) return 1;  
    return n * fact(n - 1);  
}
```

What are Anonymous Functions? (Interview Answer – 3 Lines)

An **anonymous function** is a function **without a name**.

It is usually assigned to a variable or passed as an argument to another function.

Anonymous functions are commonly used in callbacks and function expressions.